

Spring Core Framework

Java Backend Interview — Short Notes

1. Spring Framework Overview

- Spring is a comprehensive Java application framework providing infrastructure support — lets developers focus on business logic
- **Core features:** IoC/DI container, AOP, transaction management, MVC, data access, testing support
- **Spring Boot** — opinionated, auto-configured Spring. Convention over configuration. Embedded server (Tomcat). Production-ready via Actuator

Module	Purpose	Key Classes/Annotations
Spring Core	IoC container, DI, bean lifecycle, events	ApplicationContext, BeanFactory, @Bean, @Component
Spring AOP	Aspect-oriented programming — cross-cutting concerns	@Aspect, @Before, @After, @Around, Pointcut, JoinPoint
Spring MVC	Web MVC framework — REST and traditional web apps	@Controller, @RestController, @RequestMapping, DispatcherServlet
Spring Data	Data access abstraction — JPA, MongoDB, Redis, etc.	JpaRepository, CrudRepository, @Query, @Transactional
Spring Security	Authentication, authorisation, OAuth2, CSRF	SecurityFilterChain, @PreAuthorize, UserDetailsService
Spring Boot	Auto-configuration, embedded server, Actuator	@SpringBootApplication, @EnableAutoConfiguration, application.yml
Spring Cloud	Microservices patterns — gateway, discovery, config	@EnableEurekaClient, @FeignClient, @RefreshScope
Spring Batch	Batch processing — read/process/write large data	Job, Step, ItemReader, ItemProcessor, ItemWriter
Spring Integration	Enterprise integration patterns — messaging	MessageChannel, @ServiceActivator, @Transformer
Spring WebFlux	Reactive web framework — non-blocking, async	@RestController, Mono, Flux, WebClient, RouterFunction

2. IoC Container & Dependency Injection

2.1 Inversion of Control (IoC)

- Traditional: your code creates and manages dependencies. IoC: framework creates and injects dependencies — control inverted
- The Spring IoC container manages object creation, wiring, lifecycle — you declare dependencies, Spring provides them
- **BeanFactory** — basic container, lazy initialization. **ApplicationContext** — extends BeanFactory, adds AOP, events, i18n, eager init. Always prefer ApplicationContext

ApplicationContext Implementation

Use Case

Notes

AnnotationConfigApplicationContext	Standalone app with Java/annotation config	Most common in Spring Boot. Pass @Configurati on classes
ClassPathXmlApplicationContext	XML-based config — legacy	beans.xml on classpath. Rarely used in new projects
FileSystemXmlApplicationContext	XML config from filesystem path	Absolute or relative path to XML file
WebApplicationContext	Web application context — per DispatcherServlet	Root vs child context. Loaded by ContextLoade rListener
GenericWebApplicationContext	Programmatic web context setup	Used internally by Spring MVC test framework

2.2 Dependency Injection Types

1. Constructor Injection (Recommended)

```
@Service
public class OrderService {
    private final PaymentService paymentService; // final – immutable
    private final InventoryService inventoryService;
    // @Autowired optional when single constructor (Spring 4.3+)
    public OrderService(PaymentService paymentService,
        InventoryService inventoryService) {
        this.paymentService = paymentService;
        this.inventoryService = inventoryService;
    }
}
```

2. Setter Injection (Optional dependencies)

```
@Service
public class NotificationService {
    private EmailClient emailClient;
    @Autowired // optional – bean may not exist
    public void setEmailClient(EmailClient emailClient) {
        this.emailClient = emailClient;
    }
}
```

3. Field Injection (Avoid in production — hard to test)

```
@Service
public class ReportService {
    @Autowired // avoid – cannot make field final, hides dependencies
    private DataSource dataSource;
}
```

	Constructor	Setter	Field
Immutability	Yes — field can be final	No — mutable	No — cannot be final
Null safety	Cannot inject null — NPE at startup	Optional — may be null	May be null

Testability	Best — pass mocks via constructor	Good — call setter in test	Poor — needs reflection/Spring context
Circular dep	Fails fast at startup — detected early	Allowed — Spring breaks cycle	Allowed
Required deps	Yes — all constructor args required	Optional — @Autowired(required=false)	Required by default
Recommended	Yes — Spring team recommends	Only for optional dependencies	No — avoid in production

2.3 @Autowired Disambiguation

- When multiple beans of same type exist, Spring cannot choose — use qualifiers

```
// Multiple implementations of PaymentProcessor
@Component("creditCardProcessor")
public class CreditCardProcessor implements PaymentProcessor {}
@Component("paypalProcessor")
public class PaypalProcessor implements PaymentProcessor {}

// Solution 1: @Qualifier
@Autowired @Qualifier("creditCardProcessor")
private PaymentProcessor paymentProcessor;

// Solution 2: @Primary — preferred when one should be default
@Component @Primary
public class CreditCardProcessor implements PaymentProcessor {}

// Solution 3: inject all implementations
@Autowired
private List allProcessors; // Spring injects all

// Solution 4: field name matches bean name
@Autowired
private PaymentProcessor creditCardProcessor; // name matches bean
```

3. Spring Beans

3.1 Defining Beans

- **@Component** — generic stereotype. Detected by component scan
- **@Service** — business layer. Semantically meaningful — same as @Component functionally
- **@Repository** — data access layer. Also enables PersistenceExceptionTranslation
- **@Controller / @RestController** — web layer. @RestController = @Controller + @ResponseBody
- **@Bean** — method-level in @Configuration class. Full control over instantiation
- **@Configuration** — declares bean definitions. Uses CGLIB proxy to ensure singleton semantics

```
@Configuration
public class AppConfig {
    @Bean // method name = default bean name
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://localhost/mydb");
        return ds;
    }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) { // Spring injects dataSource
        return new JdbcTemplate(dataSource);
    }
}
```

```
}
}
```

3.2 Component Scanning

```
@SpringBootApplication // includes @ComponentScan for base package
public class Application { ... }

// Explicit scan:
@ComponentScan(basePackages = {"com.example.service", "com.example.repo"})
// Or by class:
@ComponentScan(basePackageClasses = {OrderService.class, UserRepo.class})
// Exclude specific classes:
@ComponentScan(excludeFilters = @ComponentScan.Filter(
    type = FilterType.ASSIGNABLE_TYPE, classes = LegacyService.class))
```

3.3 Bean Scopes

Scope	Description	Created	Use Case
singleton	ONE instance per Spring container (default). Shared across all requests	At startup (eager) or first request (lazy)	Stateless services, repositories, config
prototype	NEW instance every time bean is requested from container	On each getBean() / @Autowired injection	Stateful objects, non-thread-safe helpers
request	ONE instance per HTTP request. Destroyed when request ends	Per HTTP request	Web: request-specific data (current user, request context)
session	ONE instance per HTTP session. Lives for user session duration	Per HTTP session	Web: user-session data (shopping cart, user prefs)
application	ONE instance per ServletContext (entire web app lifetime)	At web app startup	Web: shared app-level data (app settings)
websocket	ONE instance per WebSocket session	Per WebSocket connection	WebSocket handlers with connection state

```
@Component @Scope("prototype")
public class EmailBuilder { ... }
```

■ Injecting prototype bean into singleton: Spring injects ONE prototype instance at startup — defeating prototype scope. Fix: use `ObjectFactory`, `ApplicationContext.getBean()`, or `@Lookup` method injection

```
// @Lookup – Spring overrides method to return new prototype each call
@Component
public abstract class SingletonService {
    @Lookup
    public abstract EmailBuilder getEmailBuilder(); // new instance each call
}
```

3.4 Lazy Initialization

- **@Lazy** — bean created only when first requested, not at startup
- Spring Boot 2.2+: `spring.main.lazy-initialization=true` — all beans lazy globally
- **Pros:** faster startup, less memory on startup. **Cons:** errors discovered late (first request not startup)

```
@Service @Lazy
public class HeavyService { ... } // created on first use
```

4. Bean Lifecycle

- Spring manages full lifecycle: instantiation → dependency injection → initialization → use → destruction

Phase	What Happens	Hook / Mechanism
1. Instantiation	Spring creates bean instance via constructor	Constructor called. @Autowired constructor injection here
2. Dependency Injection	Spring injects dependencies — field, setter injection	@Autowired fields/setters populated
3. BeanNameAware	If implements BeanNameAware, Spring calls setName()	setName(String name)
4. BeanFactoryAware	setBeanFactory() called — access to BeanFactory	setBeanFactory(BeanFactory factory)
5. ApplicationContextAware	setApplicationContext() called — access to context	setApplicationContext(Application Context ctx)
6. BeanPostProcessor (before)	postProcessBeforeInitialization() on ALL beans	BeanPostProcessor — AOP proxies created here
7. @PostConstruct	Init method called after all injection complete	@PostConstruct method (JSR-250). Or InitializingBean.afterPropertiesSet()
8. @Bean(initMethod)	Custom init method declared on @Bean	@Bean(initMethod = "init")
9. BeanPostProcessor (after)	postProcessAfterInitialization() on ALL beans	AOP proxy wrapping complete here
10. Bean in Use	Bean ready — injected into other beans, handles requests	Normal operation
11. @PreDestroy	Cleanup before bean destroyed — close connections, flush	@PreDestroy method. Or DisposableBean.destroy()
12. @Bean(destroyMethod)	Custom destroy method declared on @Bean	@Bean(destroyMethod = "cleanup")

```
@Component
public class DatabasePool {
    @PostConstruct
    public void init() {
        // called after all dependencies injected
        pool.initialize(); // safe — all @Autowired fields ready
    }

    @PreDestroy
    public void cleanup() {
        pool.close(); // release connections on shutdown
    }
}
```

✓ @PostConstruct vs constructor: constructor runs before DI is complete. @PostConstruct runs after ALL dependencies injected — safe to use them

■ @PreDestroy NOT called for prototype beans — Spring doesn't track prototype lifecycle after creation

5. Key Spring Annotations

5.1 Core Stereotype & Config Annotations

Annotation	Target	Purpose & Notes
@Component	Class	Generic Spring-managed bean. Base for @Service, @Repository, @Controller
@Service	Class	Business logic layer. Semantically @Component — no extra behavior currently
@Repository	Class	Data access layer. Enables exception translation (DataAccessException wrapping)
@Controller	Class	MVC controller. Returns view names. Processes @RequestMapping methods
@RestController	Class	@Controller + @ResponseBody. All methods return response body (JSON/XML)
@Configuration	Class	Declares @Bean methods. CGLIB-proxied to enforce singleton semantics
@Bean	Method	Defines a bean. Method name = bean name. Accepts other beans as params (injected)
@Autowired	Field/Constructor/Setter	Inject dependency by type. Required by default — NoSuchBeanDefinitionException if missing
@Qualifier("name")	Field/Param	Disambiguate when multiple beans of same type. Used with @Autowired
@Primary	Class/Method	Mark as preferred when multiple beans of same type — wins over others
@Value("\${key}")	Field/Param	Inject property from application.properties/yml. Supports SpEL: @Value("#{bean.prop}")
@Scope	Class/Method	Set bean scope: singleton (default), prototype, request, session, application
@Lazy	Class/Field	Delay bean creation until first use. Or delay injection of a dependency
@Profile	Class/Method	Activate bean only for specified profiles (dev, prod, test)
@Conditional	Class/Method	Register bean only when condition met. Base for Spring Boot @ConditionalOnXxx
@Import	Class	Import other @Configuration classes explicitly. Useful for modular config
@ComponentScan	Class	Enable component scanning for specified packages
@PropertySource	Class	Load properties file into Environment: @PropertySource("classpath:db.properties")

5.2 Lifecycle Annotations

```

@Component
public class CacheManager {
    @PostConstruct // JSR-250 – runs after DI complete
    public void warmCache() { loadCacheFromDB(); }

    @PreDestroy // JSR-250 – runs before bean destroyed
    public void flushCache() { cache.flush(); }
}

```

5.3 @Value & Property Injection

```

@Value("${server.port:8080}") // with default 8080 if not set
private int serverPort;

@Value("${db.hosts}") // comma-separated → List
private List dbHosts;

```

```

@Value("#{systemProperties['user.home']}") // SpEL expression
private String userHome;

@Value("#{T(Math).PI}") // SpEL – access static
private double pi;

✓ Prefer @ConfigurationProperties over @Value for multiple related properties — type-safe, validated, IDE completion
@ConfigurationProperties(prefix = "app.datasource")
public class DataSourceProperties {
    private String url;
    private String username;
    private int maxPoolSize = 10;
    // getters/setters
}

```

6. Configuration Approaches

6.1 Java Config (Recommended)

```

@Configuration
@PropertySource("classpath:application.properties")
public class AppConfig {
    @Value("${jdbc.url}") private String jdbcUrl;

    @Bean @Primary
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().url(jdbcUrl).build();
    }

    @Bean("secondaryDS") @Qualifier("secondary")
    public DataSource secondaryDataSource() { ... }
}

```

6.2 Annotation-Based Config

```

@SpringBootApplication // = @Configuration + @ComponentScan + @EnableAutoConfiguration
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

```

6.3 @Configuration vs @Component for @Bean methods

- **@Configuration** — CGLIB subclassed. Calling @Bean method from another @Bean method returns the SAME instance (singleton contract enforced)
- **@Component** with @Bean — NOT proxied (lite mode). Calling @Bean method directly creates new instance each time — breaks singleton!

```

@Configuration
public class Config {
    @Bean public A beanA() { return new A(beanB()); } // beanB() returns same instance
    @Bean public B beanB() { return new B(); }
}

@Component // lite mode – NO CGLIB proxy
public class Config {
    @Bean public A beanA() { return new A(beanB()); } // beanB() creates NEW B!
    @Bean public B beanB() { return new B(); } // different instance!
}

```

```
}
```

✓ Use `@Configuration` for bean definitions that call other `@Bean` methods. Add `proxyBeanMethods=false` for performance when not needed

7. Aspect-Oriented Programming (AOP)

- Modularise cross-cutting concerns — logging, security, transactions, caching — without scattering code
- **Spring AOP uses JDK dynamic proxy** (interface-based) or **CGLIB proxy** (class-based) — not bytecode weaving like AspectJ
- Spring AOP applies only to Spring-managed beans. Self-invocation (calling method within same bean) bypasses proxy!

7.1 AOP Key Concepts

Term	Definition	Example
Aspect	Module encapsulating a cross-cutting concern	LoggingAspect, SecurityAspect, TransactionAspect
Advice	Action taken at a join point — the actual code to run	@Before, @After, @Around, @AfterReturning, @AfterThrowing
Join Point	Point in execution where advice can be applied — always a method call in Spring AOP	service.placeOrder(), repo.save()
Pointcut	Expression selecting which join points to advise	execution(* com.example.service.*(..)) — all methods in service package
Target Object	Object being advised — the actual bean	OrderService instance before proxy wrapping
Proxy	Spring-created object wrapping target — intercepts calls	JDK Proxy (interface) or CGLIB subclass (no interface)
Weaving	Linking aspects with target objects to create advised object	Spring does runtime weaving at context startup
Introduction	Add new methods/interfaces to existing classes	@DeclareParents — rarely used in Spring AOP

7.2 Advice Types

Advice	Runs	Has ProceedingJoinPoint?	Notes
@Before	Before method execution	No	Cannot prevent method execution (use @Around for that)
@After	After method (finally — always runs)	No	Runs whether method succeeds or throws
@AfterReturning	After successful return only	No	Can access return value: @AfterReturning(returning="result")
@AfterThrowing	After exception thrown only	No	Can access exception: @AfterThrowing(throwing="ex")
@Around	Wraps entire method — before and after	Yes	Most powerful. Must call pjp.proceed() to invoke target. Can modify args/return

7.3 Aspect Example


```

@Aspect @Component
public class LoggingAspect {
    // Pointcut – all public methods in service package
    @Pointcut("execution(public * com.example.service..*(..))")
    public void serviceLayer() {}
    @Before("serviceLayer()")
    public void logBefore(JoinPoint jp) {
        log.info("Calling: {} with args: {}", jp.getSignature(), jp.getArgs());
    }
    @Around("serviceLayer()")
    public Object logExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed(); // MUST call proceed – invokes actual method
        long elapsed = System.currentTimeMillis() - start;
        log.info("{} took {}ms", pjp.getSignature(), elapsed);
        return result;
    }
    @AfterThrowing(pointcut="serviceLayer()", throwing="ex")
    public void logException(JoinPoint jp, Exception ex) {
        log.error("Exception in {}: {}", jp.getSignature(), ex.getMessage());
    }
}

```

7.4 Pointcut Expressions

```

// All methods in a package (and sub-packages)
execution(* com.example.service..*(..))
// Specific method – any return type, any args
execution(* com.example.service.OrderService.placeOrder(..))
// Methods annotated with @Transactional
@annotation(org.springframework.transaction.annotation.Transactional)
// Methods in classes annotated with @Service
within(@org.springframework.stereotype.Service *)
// Combine with &&, ||, !
execution(* com.example..*(..)) && !execution(* com.example.util..*(..))

```

■ AOP self-invocation bypass: if method A() calls method B() in SAME bean, @Around on B() is NOT triggered — A() calls real B() directly, not through proxy. Fix: inject self (ApplicationContext.getBean()) or restructure to different bean

8. Spring Event System

- Built-in publish-subscribe mechanism for loose coupling within the application context
- **ApplicationEvent** — base class for events (or use any POJO in Spring 4.2+)
- **ApplicationEventPublisher** — publish events. Inject via @Autowired
- **@EventListener** — annotate method to listen for events
- By default **synchronous** — listener runs in same thread as publisher. Use @Async for async listeners
- **@TransactionalEventListener** — listen for events tied to transaction phase (AFTER_COMMIT, AFTER_ROLLBACK, AFTER_COMPLETION, BEFORE_COMMIT)

```

// Event class (POJO – Spring 4.2+)
public record OrderPlacedEvent(Long orderId, String userEmail) {}

// Publisher
@Service

```

```

public class OrderService {
    @Autowired private ApplicationEventPublisher publisher;

    public void placeOrder(Order order) {
        orderRepo.save(order);
        publisher.publishEvent(new OrderPlacedEvent(order.getId(), order.getEmail()));
    }
}

// Listener
@Component
public class EmailNotificationListener {
    @EventListener
    @Async // run in separate thread
    public void handleOrderPlaced(OrderPlacedEvent event) {
        emailService.sendConfirmation(event.userEmail(), event.orderId());
    }

    // Transactional – only fires after transaction commits
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void handleAfterCommit(OrderPlacedEvent event) { ... }
}

```

• **Built-in Application Events:** ContextRefreshedEvent (context initialized/refreshed), ContextStartedEvent, ContextStoppedEvent, ContextClosedEvent, ApplicationStartedEvent, ApplicationReadyEvent

9. Spring Profiles & Environment

9.1 @Profile

- Activate beans conditionally based on environment — dev, test, staging, prod

```

@Configuration @Profile("dev")
public class DevDataSourceConfig {
    @Bean public DataSource dataSource() { return new H2DataSource(); }
}

@Configuration @Profile("prod")
public class ProdDataSourceConfig {
    @Bean public DataSource dataSource() { return new HikariDataSource(); }
}

// Activate profiles:
spring.profiles.active=prod # application.yml
SPRING_PROFILES_ACTIVE=prod # environment variable
java -jar app.jar --spring.profiles.active=prod # command line
@ActiveProfiles("test") // in @SpringBootTest

```

9.2 Profile-specific Properties

```

application.yml # base config – always loaded
application-dev.yml # overrides for dev profile
application-prod.yml # overrides for prod profile
application-test.yml # overrides for test profile

# application.yml – multi-document YAML (Spring Boot 2.4+)
spring.application.name: my-app
---
spring.config.activate.on-profile: prod
spring.datasource.url: jdbc:postgresql://prod-server/mydb

```

9.3 Environment & @Conditional

- **Environment** — access to profiles, properties, system properties. Inject via `@Autowired`
- **@Conditional** — register bean only when condition (Condition interface) is true
- **Spring Boot @ConditionalOnXxx** shortcuts:
 - `@ConditionalOnProperty(name="feature.flag", havingValue="true")` — only if property set
 - `@ConditionalOnClass(Redis.class)` — only if class on classpath
 - `@ConditionalOnMissingBean(DataSource.class)` — only if no DataSource bean defined
 - `@ConditionalOnBean(AuthProvider.class)` — only if AuthProvider bean exists
 - `@ConditionalOnWebApplication` — only in web application context

```
@Bean @ConditionalOnProperty(name="cache.type", havingValue="redis")
public CacheManager redisCacheManager() { ... }

@Autowired Environment env;
env.getActiveProfiles(); // current active profiles
env.getProperty("server.port"); // read any property
env.acceptsProfiles("prod"); // check if profile active
```

10. Spring Boot Auto-Configuration

- Spring Boot automatically configures beans based on classpath contents and properties — convention over configuration
- Triggered by `@EnableAutoConfiguration` (included in `@SpringBootApplication`)
- Auto-config classes listed in **META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports** (Spring Boot 2.7+) or `spring.factories`
- Each auto-config class uses `@ConditionalOnXxx` to only activate when appropriate

```
// Example: DataSourceAutoConfiguration activates when:
// 1. DataSource class on classpath (spring-jdbc present)
// 2. No DataSource bean already defined (@ConditionalOnMissingBean)
// 3. spring.datasource.url property set

// See all auto-config applied:
java -jar app.jar --debug # prints conditions evaluation report
// In tests:
@SpringBootTest
@AutoConfigureTestDatabase(replace = Replace.NONE)
```

Auto-Config Class	Activates When	Configures
DataSourceAutoConfiguration	spring-jdbc on classpath, no DataSource bean	HikariCP DataSource from spring.datasource.*
JpaRepositoriesAutoConfiguration	spring-data-jpa on classpath	JPA repositories, EntityManagerFactory, TransactionManager
SecurityAutoConfiguration	spring-security on classpath	Basic security with random password, form login
WebMvcAutoConfiguration	spring-webmvc, not WebFlux, no WebMvcConfigurer	DispatcherServlet, ContentNegotiation, MessageConverters
KafkaAutoConfiguration	spring-kafka on classpath	KafkaTemplate, KafkaAdmin from spring.kafka.*
RedisAutoConfiguration	lettuce/jedis on classpath	RedisTemplate, StringRedisTemplate from spring.redis.*

ActuatorAutoConfiguration	spring-boot-actuator on classpath	Health, Metrics, Info endpoints at /actuator/*
----------------------------------	-----------------------------------	--

```
// Exclude specific auto-config
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
// Or in properties:
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

11. Transaction Management

11.1 @Transactional

- Spring manages transactions declaratively — no boilerplate begin/commit/rollback code
- Works via AOP proxy — calling @Transactional method from OUTSIDE the bean starts transaction. Self-invocation bypasses it!
- Default: rollback on **RuntimeException and Error**. Does NOT rollback on checked exceptions by default

```
@Service
public class OrderService {
    @Transactional // begins TX before, commits after, rolls back on RuntimeException
    public Order placeOrder(OrderRequest req) {
        Order order = orderRepo.save(new Order(req));
        paymentService.charge(req.getPayment()); // if throws → rollback
        inventoryService.reserve(req.getItems()); // if throws → rollback
        return order;
    }
    @Transactional(readOnly = true) // hint to DB driver – no writes, allow optimisations
    public Order findById(Long id) { return orderRepo.findById(id).orElseThrow(); }
}
```

11.2 Transaction Attributes

Attribute	Description	Example
propagation	How TX relates to existing TX	REQUIRED (default), REQUIRES_NEW, SUPPORTS, MANDATORY, NEVER, NOT_SUPPORTED, NESTED
isolation	DB isolation level — what concurrent TX can see	DEFAULT, READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE
timeout	Max seconds for TX — rolls back if exceeded	@Transactional(timeout = 30)
readOnly	Hint — no writes. Enables optimisations (Hibernate flush mode NEVER)	@Transactional(readOnly = true)
rollbackFor	Rollback on these exception types (including checked)	@Transactional(rollbackFor = IOException.class)
noRollbackFor	Don't rollback on these exception types	@Transactional(noRollbackFor = NotFoundException.class)

11.3 Propagation Types

Propagation	Behaviour	Use Case
-------------	-----------	----------

REQUIRED	Join existing TX. If none, create new. (Default)	Most service methods
REQUIRES_NEW	Suspend existing TX. Always create new TX. Commits independently	Audit logging — must persist even if outer TX rolls back
SUPPORTS	Join if TX exists; run without TX if none	Read methods that work with or without TX
MANDATORY	Must join existing TX. Exception if no TX	Internal methods that assume TX exists
NEVER	Must NOT have TX. Exception if TX exists	Methods that must not run in TX
NOT_SUPPORTED	Suspend existing TX, run without TX. Resume after	Operations that don't support TX
NESTED	Nested TX (savepoint) — rolls back to savepoint on failure, outer continues	Partial rollback — try operation, rollback only if on failure

12. Spring Expression Language (SpEL)

- Powerful expression language for querying and manipulating objects at runtime
- Used in: @Value, Spring Security (@PreAuthorize), Spring Data queries, @Cacheable keys, @Conditional

Expression Type	Syntax	Example
Property access	object.property	<code>#{user.name}</code> — access name property
Method call	object.method(args)	<code>#{list.size()}</code> — call size() method
Static field/method	T(class).fieldOrMethod	<code>#{T(Math).PI}</code> , <code>#{T(System).currentTimeMillis()}</code>
Arithmetic	+, -, *, /, %, ^	<code>#{2 + 3}</code> , <code>#{10 / 2}</code>
Comparison	==, !=, <, >, <=, >=, lt, gt	<code>#{user.age >= 18}</code>
Logical	&&, , !, and, or, not	<code>#{user.active and user.verified}</code>
Ternary/Elvis	condition ? a : b or a ?: b	<code>#{name != null ? name : 'Guest'}</code> , <code>#{name ?: 'Guest'}</code>
Collection filter	list.[condition]	<code>#{employees.[salary > 50000]}</code> — filter list
Collection project	list.![property]	<code>#{employees.![name]}</code> — extract name from each
Safe navigation	object?.property	<code>#{user?.address?.city}</code> — null-safe chain
Bean reference	@beanName	<code>#{@myService.calculate()}</code> — call Spring bean method
System properties	systemProperties['key']	<code>#{systemProperties[user.home]}</code>

```
// @Value with SpEL
@Value("#{systemProperties['user.timezone'] ?: 'UTC'}")
private String timezone;

// Spring Security – method security
@PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
public User getUser(Long userId) { ... }

// @Cacheable with SpEL key
@Cacheable(value="users", key="#userId + '-' + #region")
public User getUser(Long userId, String region) { ... }
```

13. Frequently Asked & Tricky Interview Questions

Q1. What is IoC? How does Spring implement it?

→ Inversion of Control: framework manages object creation and lifecycle instead of application code. Spring implements via the IoC container (ApplicationContext) — you declare beans and dependencies, Spring creates and wires them. DI is the mechanism — constructor, setter, or field injection

Q2. BeanFactory vs ApplicationContext?

→ BeanFactory: basic container, lazy initialization, no AOP, no events, no i18n. ApplicationContext: extends BeanFactory, adds eager initialization, AOP support, event publishing, i18n, @PostConstruct/@PreDestroy support. Always use ApplicationContext

Q3. Constructor vs setter vs field injection? Which is best?

→ Constructor injection: recommended — immutable fields (final), all deps required, fails fast, easy to test with mocks. Setter: for optional deps. Field (@Autowired directly): avoid — hides deps, hard to test, can't be final, requires Spring context for tests

Q4. What happens if Spring cannot find a bean to inject?

→ Throws NoSuchBeanDefinitionException at startup. Fix: ensure bean is @Component/@Bean and within component scan. Use @Autowired(required=false) for optional deps. Or use Optional injection. Or @Nullable annotation

Q5. How does @Qualifier work with @Autowired?

→ @Qualifier("beanName") disambiguates when multiple beans of same type exist. Used alongside @Autowired. Alternative: @Primary marks preferred bean. Or match field name to bean name. Custom qualifiers: create annotation annotated with @Qualifier

Q6. What is @Primary? When would you use it?

→ Marks a bean as preferred when multiple beans of same type exist — wins without @Qualifier. Use when one implementation should be default and others are special cases. @Qualifier always overrides @Primary

Beans & Scopes

Q7. What are Spring bean scopes? Which is default?

→ Singleton (default): one instance per container, shared. Prototype: new instance per injection/getBean(). Request/Session/Application/WebSocket: web-scoped. Singleton is thread-unsafe if bean holds mutable state — design stateless or use prototype

Q8. What is the problem with injecting a prototype bean into a singleton?

→ Spring injects one prototype instance at singleton creation — same instance reused every call (defeats prototype purpose). Fix: inject ObjectProvider, use @Lookup method injection, or call applicationContext.getBean() each time

Q9. @PostConstruct vs constructor for initialization. What's the difference?

→ Constructor: runs before DI — injected fields are NULL. @PostConstruct: runs after ALL injection complete — safe to use @Autowired fields. Use @PostConstruct for initialization that depends on injected beans

Q10. What is @Lazy? When to use it?

→ @Lazy delays bean creation until first request. Use: heavy beans rarely needed, beans with slow initialization (DB connections, remote calls), break circular dependency workaround. spring.main.lazy-initialization=true makes all beans lazy globally

Q11. How does Spring handle circular dependencies?

→ Circular deps in constructor injection: fails at startup (correct — fix by redesigning). Circular deps in field/setter injection: Spring resolves by creating a partially-initialized bean first (using 3-stage singletonObjects/earlySingletonObjects/singletonFactories). Spring Boot 2.6+ disables this by default

Q12. What is the difference between @Component, @Service, @Repository?

→ Functionally identical — all register as Spring beans. Semantic difference: @Repository enables PersistenceExceptionTranslation (wraps DB exceptions in DataAccessException). @Service is a marker for business layer. @Component is generic. Use correct stereotype for clarity

Configuration

Q13. What is @Configuration? Why use CGLIB proxy?

→ @Configuration marks a class as source of bean definitions. Spring uses CGLIB to subclass it — when @Bean method A calls @Bean method B, CGLIB intercepts and returns existing singleton B instead of creating a new one. Without CGLIB (lite

mode with `@Component`), each call creates a new instance

Q14. What is the difference between `@Bean` in `@Configuration` vs `@Component`?

→ `@Configuration` + `@Bean`: CGLIB proxy — singleton semantics enforced. `@Bean` method calls return same singleton.
`@Component` + `@Bean` (lite mode): no proxy — each `@Bean` method call creates new instance. Use `@Configuration` when `@Bean` methods call each other

Q15. How do you import properties in Spring?

→ `@PropertySource("classpath:db.properties")` on `@Configuration` class. Then `@Value("${key}")` to inject. Spring Boot auto-loads `application.properties/application.yml`. Profile-specific: `application-{profile}.yml` overrides base config

Q16. `@Value` vs `@ConfigurationProperties`?

→ `@Value`: single property injection with SpEL. `@ConfigurationProperties`: bind group of related properties to typed POJO — type-safe, validated (`@Validated`), IDE completion, relaxed binding (camelCase/kebab-case/UPPER_CASE all work). Prefer `@ConfigurationProperties` for related config groups

AOP

Q17. What is AOP? What problems does it solve?

→ Aspect-Oriented Programming modularises cross-cutting concerns — logging, security, transactions, caching — that would otherwise be scattered across many classes. Spring AOP uses proxies (JDK/CGLIB) to intercept method calls and apply advice

Q18. What is the difference between Spring AOP and AspectJ?

→ Spring AOP: proxy-based, runtime weaving, applies only to Spring beans, supports method join points only. AspectJ: full AOP framework, compile-time/load-time weaving, applies to any Java code, supports field/constructor join points. Spring AOP uses AspectJ annotations but not AspectJ weaver

Q19. What is the self-invocation problem in AOP?

→ If method A() calls method B() in same bean, B() is called on 'this' — bypasses the Spring proxy. `@Around/@Before` on B() won't trigger. Fix: inject self via `@Autowired`, use `ApplicationContext.getBean()`, or restructure so B() is in a different bean

Q20. JDK Dynamic Proxy vs CGLIB in Spring AOP?

→ JDK Proxy: requires target to implement interface — creates proxy implementing same interface. CGLIB: creates subclass of target — works for classes without interface. Spring uses JDK proxy if interface available, CGLIB otherwise. Spring Boot defaults to CGLIB always (`spring.aop.proxy-target-class=true`)

Q21. `@Before` vs `@Around` advice?

→ `@Before`: runs before method, cannot stop execution or modify args/return. `@Around`: wraps entire method, must call `pjp.proceed()`, can modify args, return value, catch exceptions. Most powerful. Use `@Around` when you need to modify return or conditionally prevent execution

Q22. What is `@EnableAspectJAutoProxy`?

→ Enables Spring AOP — tells Spring to look for `@Aspect` beans and create proxies. In Spring Boot, auto-configured when `spring-boot-starter-aop` is on classpath. `proxyTargetClass=true` forces CGLIB even for interfaces

Events & Profiles

Q23. What is Spring Event system? How is it used?

→ Built-in pub-sub within `ApplicationContext`. Publisher injects `ApplicationEventPublisher` and calls `publishEvent()`. Listeners use `@EventListener` on methods. Synchronous by default. `@Async` for async. Decouples components without external broker

Q24. What is `@TransactionalEventListener`?

→ Event listener bound to transaction phase — `AFTER_COMMIT` (default), `AFTER_ROLLBACK`, `AFTER_COMPLETION`, `BEFORE_COMMIT`. Use to send emails/notifications only after DB commit — ensures data is persisted before notifying

Q25. What are Spring Profiles? How do you activate them?

→ Profiles group beans/config for specific environments. Activate via: `spring.profiles.active` property, `SPRING_PROFILES_ACTIVE` env var, command-line arg, `@ActiveProfiles` in tests, `SpringApplication.setAdditionalProfiles()`. Profile-specific yml: `application-{profile}.yml`

Q26. What is `@ConditionalOnProperty`?

→ Spring Boot — register bean only when specific property has specific value. Example:
@ConditionalOnProperty(name='feature.cache', havingValue='redis'). matchIfMissing=true — activate even if property not set.
Foundation of Spring Boot auto-configuration

Spring Boot

Q27. What does @SpringBootApplication contain?

→ @SpringBootApplication = @Configuration (declares beans) + @ComponentScan (scans base package) + @EnableAutoConfiguration (triggers auto-config). Can customise: exclude auto-configs, change scan base packages

Q28. How does Spring Boot auto-configuration work?

→ Auto-config classes listed in AutoConfiguration.imports. Each uses @ConditionalOnXxx to activate only when appropriate (class on classpath, property set, no user-defined bean). Order controlled by @AutoConfigureBefore/@AutoConfigureAfter

Q29. How do you disable a specific auto-configuration?

→ @SpringBootApplication(exclude={DataSourceAutoConfiguration.class}) or spring.autoconfigure.exclude property. Use --debug flag to see conditions evaluation report showing why each auto-config was applied/skipped

Q30. What is spring.factories? (Spring Boot < 2.7)

→ META-INF/spring.factories — declares auto-configuration classes for Spring Boot to load. Key: org.springframework.boot.autoconfigure.EnableAutoConfiguration. Spring Boot 2.7+ uses META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports instead

Transactions

Q31. What is @Transactional? How does Spring implement it?

→ Declarative transaction management via AOP proxy. Spring creates proxy around bean — before method call, begins TX; after successful return, commits; on RuntimeException, rolls back. Must be called through proxy (not self-invocation) to work

Q32. Default rollback behaviour of @Transactional?

→ Rolls back on RuntimeException and Error. Does NOT rollback on checked exceptions by default. Override with rollbackFor=IOException.class. Or noRollbackFor=NotFoundException.class to prevent rollback on specific RuntimeException

Q33. What is REQUIRES_NEW propagation?

→ Suspends existing transaction, starts new independent transaction. New TX commits/rolls back independently of outer TX. Use for: audit logging, outbox events — must persist even if business TX rolls back

Q34. What is readOnly=true in @Transactional?

→ Hint to the persistence layer and DB driver — no writes expected. Hibernate: sets flush mode to NEVER (skips dirty checking at commit). DB: may use read replica, skip locks. Performance optimization for read-only operations

Q35. Why doesn't @Transactional work on private methods?

→ Spring AOP creates a proxy that wraps the bean — but proxy can only intercept public methods. Private methods are called directly on 'this', bypassing proxy. Solution: make methods public or use AspectJ weaving for non-proxy interception

Tricky & Advanced

Q36. What is the difference between @Autowired and @Inject and @Resource?

→ @Autowired: Spring-specific, by type first, @Qualifier for name. @Inject: JSR-330 standard, identical to @Autowired, @Named for qualifier. @Resource: JSR-250, by name first, then type. @Autowired is standard in Spring projects

Q37. What is ObjectProvider and ObjectFactory?

→ ObjectProvider: lazy-resolved, optional dependencies. getIfAvailable() returns null if no bean. getObject() throws if missing. Supports getting multiple. ObjectFactory: simple lazy lookup — getObject() each time creates new (for prototype). Use to avoid circular deps or lazy init

Q38. What is @DependsOn?

→ Forces specified beans to initialize before this bean — for implicit dependencies not visible in code (static resource init, legacy frameworks). @DependsOn("databaseMigrator") ensures migrator runs before service that needs migrated schema

Q39. What is BeanPostProcessor?

→ Interface to intercept bean initialization globally — applied to ALL beans. postProcessBeforeInitialization() runs before @PostConstruct. postProcessAfterInitialization() runs after. Spring AOP proxy creation is a BeanPostProcessor. Use for:

custom annotations, cross-cutting initialization

Q40. What is @ImportResource?

→ Import legacy XML bean definitions into Java config: `@ImportResource("classpath:legacy-beans.xml")`. Allows mixing XML and Java configuration during migration

Q41. What is ApplicationContextAware?

→ Interface — Spring calls `setApplicationContext()` on bean that implements it, giving direct access to the `ApplicationContext`. Use to programmatically get beans: `context.getBean()`. Generally prefer proper `@Autowired` injection; use this only when necessary

Q42. What is SmartInitializingSingleton?

→ `afterSingletonsInstantiated()` called after ALL singleton beans initialized. Useful for validation or setup that requires all beans to be ready. Safer than `@PostConstruct` for inter-bean operations

Q43. Explain Spring bean circular dependency error and fix

→ CIRCULAR DEPENDENCY: A needs B, B needs A — constructor injection fails at startup (correct). Spring Boot 2.6+ disables field/setter circular dep too. Fix: redesign (extract common dependency C), use `@Lazy` on one injection point, setter injection, or `ApplicationContext.getBean()`

Q44. What is @Scope(proxyMode = ScopedProxyMode.TARGET_CLASS)?

→ Creates a proxy for shorter-scoped beans (request/session) injected into singletons. Without proxy: singleton holds stale request-scoped bean. With proxy: singleton holds proxy that delegates to correct request-scoped instance at call time

Q45. How do you create a custom Spring Boot starter?

→ 1. Create auto-config class with `@ConditionalOnXxx`. 2. Register in `META-INF/spring/AutoConfiguration.imports`. 3. Package as jar. 4. Add `spring-boot-autoconfigure` dependency. Consumer just adds your jar — auto-config activates automatically. Optionally add `spring-boot-configuration-processor` for IDE support

Q46. What is @RefreshScope (Spring Cloud)?

→ Beans marked `@RefreshScope` are recreated when `/actuator/refresh` is called. Used for beans that read config at startup — calling refresh re-reads config and re-creates bean with new values. Used with Spring Cloud Config for live config updates

Q47. What is the Environment abstraction in Spring?

→ Unified interface for property sources and profiles. Sources in order (highest priority first): system properties, OS env vars, application.properties, default properties. Access via `Environment` bean or `@Value`. `@PropertySource` adds custom sources

Q48. What happens during SpringApplication.run()?

→ 1. Create `SpringApplication`. 2. Determine application type (web/reactive/none). 3. Load `ApplicationContextInitializers` and `ApplicationListeners` from `spring.factories`. 4. Create `ApplicationContext`. 5. Refresh (component scan, auto-config, bean init). 6. Publish `ApplicationStartedEvent`. 7. Run `ApplicationRunners/CommandLineRunners`

Q49. What is @EnableAutoConfiguration(exclude=...)?

→ Disables specific auto-configuration classes. Used when auto-config conflicts with custom setup. Same as `spring.autoconfigure.exclude` property. Common: exclude `DataSourceAutoConfiguration` when using multiple datasources or embedding in larger app

Q50. How do you write a custom @Conditional?

→ Implement `Condition` interface: `boolean matches(ConditionContext ctx, AnnotatedTypeMetadata meta)`. Access environment, classpath, beans in ctx. Use `@Conditional(MyCondition.class)` on `@Bean` or `@Configuration`. Spring Boot's `@ConditionalOnXxx` all do this internally